

AULA 02 · UC1 DESENVOLVIMENTO DE SISTEMAS

Da Nuvem ao Código

GitHub Codespaces, Console Turbo & Variáveis

Criando seu primeiro repositório, configurando o ambiente na nuvem e dominando a saída e armazenamento de dados em JavaScript.

Docente: Jeronimo Silva · Senac Centro · Tarde (2026)

ROTEIRO PEDAGÓGICO

O Que Faremos Hoje

1. Do GitHub ao Codespaces

Criar o primeiro repositório e subir um computador completo na nuvem em 1 clique.

2. Terminal & Node.js

Entender o terminal integrado, o interpretador Node e a execução de arquivos `.js`.

3. console.log Turbo

Interpolação de texto e alinhamento profissional com `repeat`, `padStart` e `padEnd`.

4. Gavetas de Memória

Variáveis e constantes: `const` vs `let`, tipos primitivos e o operador `typeof`.

Bônus de Interatividade: Instalação do módulo `@inquirer/prompts` para capturar respostas pelo teclado.

Termos e Ferramentas da Aula

Conceito	O que significa na prática?
GitHub Codespaces	Nosso computador Linux completo na nuvem, com VS Code e Node prontos.
Node.js	O motor que lê e executa nosso código JavaScript fora do navegador.
<code>console.log()</code>	O comando que imprime dados e mensagens na tela preta do terminal.
Template Literal	Texto entre crases (``) que permite embutir variáveis via <code>\${...}</code> .
<code>repeat()</code>	Método de texto que replica caracteres várias vezes (bordas e divisórias).
<code>padStart</code> / <code>End</code>	Métodos para alinhar textos à direita ou esquerda (recibos e tabelas).
<code>const</code> / <code>let</code>	Caixas na memória: <code>const</code> é fixa (padrão) e <code>let</code> pode mudar.
<code>typeof</code>	Operador que pergunta à máquina qual é o tipo de dado de uma variável.

Passo 1: Criando o Primeiro Repositório

Ontem você criou sua conta no GitHub. Agora vamos criar sua pasta oficial de estudos:

1. Acesse github.com e faça login na sua conta.
2. No canto superior direito, clique no ícone  → **New repository**.
3. **Repository name**: digite `senac-programador-sistemas` (ou `programador-de-sistemas`).
4. Marque a opção **Public** (Público).
5. Marque a caixinha: **Add a README file** (*fundamental!!*).
6. Clique no botão verde: **Create repository**.

 **Dica:** Este repositório será o seu **caderno e portfólio oficial** durante todo o curso! Ele acompanhará você nas aulas de lógica, banco de dados e testes.

Passo 2: Criando o GitHub Codespaces

Não precisamos instalar nada na máquina do laboratório. Vamos ligar uma máquina na nuvem:

Como Iniciar

1. Dentro do repositório, clique no botão verde  **Code**.
2. Clique na aba **Codespaces**.
3. Clique em **Create codespace on main**.
4. Aguarde ~30 segundos enquanto a nuvem prepara o seu VS Code completo no navegador.

Por que usar Codespaces?

- **Zero configuração:** Node.js e Git já vêm instalados.
- **Ambiente blindado:** não depende do Windows do laboratório.
- **Portabilidade:** continue programando em casa exatamente do ponto onde parou.

Conhecendo o Codespaces & Testando o Node

Quando a interface abrir no navegador, observe as 3 áreas fundamentais:

```
+-----+-----+
| ARQUIVOS (Explorador) | EDITOR DE CÓDIGO (Onde você digita) |
| [README.md]          |                               |
|                       |                               |
+-----+-----+
| TERMINAL INTEGRADO    | $ node -v                      |
|                       | v20.x.x (Node instalado e pronto!) |
+-----+-----+
```

1. Se o terminal não estiver visível, abra com o atalho: **Ctrl + `** (ou menu *Terminal* → *New Terminal*).
2. Digite o comando de teste: **node -v** e aperte **Enter**.

Criando seu Arquivo `aula02.js`

1. Na barra lateral esquerda (Explorer), clique no ícone **New File** (Novo Arquivo).
2. Nomeie o arquivo como: `aula02.js` (sempre com a extensão `.js`).
3. Digite a sua primeira instrução:

```
console.log("Olá, mundo! Meu primeiro código no Codespaces!");
```

4. No terminal, execute o programa chamando o Node:

```
node aula02.js
```

⚡ **Dica de Produtividade:** O Node possui o modo `--watch`, que reexecuta o script toda vez que você salva: `node --watch aula02.js`

O Poder do `console.log()`

O `console.log()` aceita múltiplos valores separados por vírgula e respeita seus formatos:

```
// Imprimindo vários dados juntos:  
console.log("Instrutor:", "Jeronimo", "Ano:", 2026);  
  
// Fazendo contas diretamente na saída:  
console.log("Resultado da soma 10 + 25 =", 10 + 25);
```

Template Literals (Interpolação com Crase)

A forma profissional e elegante do JavaScript moderno de juntar texto e dados:

```
const turma = "Programador de Sistemas";  
const alunos = 9;  
  
console.log(`Turma: ${turma} | Alunos presentes: ${alunos}`);
```

Criando Bordas com `repeat()`

Em vez de digitar traços manualmente, usamos o método `.repeat(quantidade)`:

```
// Cria uma linha divisória perfeita de 40 caracteres:  
console.log("=".repeat(40));  
console.log("      SISTEMA SENAC DE COMPRAS      ");  
console.log("=".repeat(40));  
  
// Divisor pontilhado:  
console.log("-".repeat(40));
```

Resultado no Terminal:

```
=====
      SISTEMA SENAC DE COMPRAS
=====
-----
```

Alinhamento com `padEnd()` e `padStart()`

Como fazer colunas alinhadas perfeitamente como em um cupom fiscal?

`texto.padEnd(tamanho, " ")`

Alinha o texto à **esquerda** e preenche o restante com espaços ou caracteres até atingir a largura.

Ideal para nomes de itens e colunas.

`texto.padStart(tamanho, " ")`

Alinha o texto à **direita** empurrando com preenchimento no início.

Ideal para preços, quantidades e totais.

```
const item = "Café Expresso";
const preco = "R$ 6,50";

// O item terá 25 caracteres (preenchido com pontos) e o preço terá 10 caracteres:
console.log(item.padEnd(25, ".") + preco.padStart(10, " "));
// Resultado: Café Expresso..... R$ 6,50
```

Mini-Desafio: O Cupom Fiscal no Terminal

Crie um arquivo chamado `recibo.js` e use `repeat`, `padStart` e `padEnd` para desenhar o seguinte recibo:

```
=====
                MERCADINHO SENAC LTDA
=====
ITEM                                VALOR
-----
1. Teclado Mecânico.....      R$ 180,00
2. Mouse Sem Fio.....          R$ 75,00
3. Cabo HDMI 2.0.....          R$ 25,00
-----
TOTAL GERAL.....              R$ 280,00
=====
```

Dica: Padronize a largura total em 40 caracteres. Use `padEnd(27, ".")` para a descrição e `padStart(13, " ")` para o preço!

O Que é uma Variável?

Na aula de ontem vimos que o computador processa dados na memória RAM. Uma **variável** é uma **caixa etiquetada** nessa memória:

Memória RAM:

```
+-----+
| [nomeAluno] |
| "Lucas"     |
+-----+
```

1. Nome da Caixa: rótulo que identifica a gaveta.
2. Conteúdo: o valor guardado lá dentro.
3. Tipo: o formato do dado (texto, número, etc).

Em JavaScript moderno, declaramos nossas caixas de duas formas:

- **const** (*Constante*): O valor é gravado e **NUNCA PODE MUDAR**.
- **let** (*Variável*): O valor pode ser **alterado/reatribuído** ao longo do programa.

A Regra de Ouro: `const` por Padrão!

★ Regra Prática: Use sempre `const` ! Só use `let` quando você tiver certeza absoluta de que aquele valor precisará mudar (como contadores ou acumuladores).

```
// Constante: dados fixos e seguros
const pi = 3.14159;
const cpf = "123.456.789-00";
// cpf = "999.999.999-99"; // ERRO! TypeError: Assignment to constant variable.

// Let: valor dinâmico
let tentativas = 0;
tentativas = tentativas + 1; // OK! Agora vale 1.
```

Por que isso importa? Usar `const` evita que você ou outro programador sobrescreva dados acidentalmente no meio do sistema!

Os Três Tipos Primitivos Fundamentais

```
// 1. STRING (Texto delimitado por aspas simples, duplas ou crase)  
const aluno = "Natanael";  
const cidade = 'Fortaleza';  
  
// 2. NUMBER (Números inteiros e decimais - SEM aspas!)  
const idade = 20;  
const preco = 49.90; // Em programação usamos PONTO (.) e não vírgula!  
  
// 3. BOOLEAN (Verdadeiro ou Falso - a chave liga/desliga da lógica)  
const cursoAtivo = true;  
const temPendencia = false;
```

⚠️ Atenção: "20" com aspas é texto (string). 20 sem aspas é número (number).

Inspecionando Tipos com `typeof`

Quando você estiver em dúvida sobre qual dado está dentro da caixa, pergunte ao JavaScript usando o operador `typeof`:

```
const matricula = "573";  
const salario = 2500.50;  
const aprovado = true;  
  
console.log(typeof matricula); // "string"  
console.log(typeof salario);   // "number"  
console.log(typeof aprovado);  // "boolean"
```

 O Teste do Trapaceiro: `typeof "100"` retorna `string` ! Se somarmos `"100" + 50`, o resultado vira texto colado: `"10050"`.

Caça aos Nomes e Tipos

Analise as declarações abaixo e diga se o JavaScript vai aceitar ou dar **ERRO**:

```
const lnome = "Gabriel";
```

```
const nome-aluno = "Caio";
```

```
let preco = 12.50;
```

```
const let = 10;
```

```
let total_pontos = 100;
```

```
const valorFinal = 50;
```

1. Quais dessas declarações geram erro de sintaxe imediatamente?
2. Por que `const let = 10;` não funciona? (*Palavra reservada*).
3. Se fizermos `valorFinal = 70;`, o que acontece ao rodar com `node` ?

O Próximo Nível: Perguntando ao Usuário

Até agora nossos scripts apenas exibem dados estáticos. E se quisermos que o programa **faça perguntas** no terminal?

Para isso usamos uma biblioteca externa chamada `@inquirer/prompts`.

```
Nosso Programa JS  ==== Pergunta ====>  Usuário no Teclado  
Nosso Programa JS  <=== Resposta =====  Usuário digita e dá Enter
```

Vamos aprender agora como inicializar um projeto Node no Codespaces e instalar pacotes do ecossistema NPM!

Instalando o Inquirer no Codespaces

Abra o terminal integrado (`Ctrl + ``) e rode os 3 comandos abaixo:

```
# 1. Cria o arquivo de configuração do projeto (package.json):
```

```
npm init -y
```

```
# 2. Habilita o JavaScript moderno com módulos (import / await):
```

```
npm pkg set type="module"
```

```
# 3. Baixa e instala a biblioteca de perguntas interativas:
```

```
npm install @inquirer/prompts
```

 O que aconteceu? A pasta `node_modules` e o arquivo `package.json` foram criados com o Inquirer pronto para uso!

Nosso Primeiro Script Interativo (`app_interativo.js`)

Crie o arquivo `app_interativo.js` e veja como a captura de texto simples funciona:

```
import { input } from '@inquirer/prompts';

console.log("=".repeat(40));
console.log("      SISTEMA DE CADASTRO SENAC      ");
console.log("=".repeat(40));

// O comando "await input" aguarda a digitação do usuário:
const nome = await input({ message: "Digite seu nome completo:", required: true });
const curso = await input({ message: "Qual curso você está fazendo?" });

console.log("\n" + "-".repeat(40));
console.log(`Sucesso! Bem-vindo(a), ${nome}!`);
console.log(`Matriculado com sucesso em: ${curso}.`);
console.log("-".repeat(40));
```

Execute com: `node app_interativo.js` e responda no terminal!

1. `input()` · Entrada de Texto

Usado para capturar qualquer informação digitada livremente pelo usuário:

```
import { input } from '@inquirer/prompts';

const nome = await input({
  message: 'Digite seu nome completo:',
  required: true // Impede o usuário de deixar em branco e dar Enter!
});

console.log(`Olá, ${nome}! Seja bem-vindo(a) ao Senac.`);
```

 **Retorno:** Devolve sempre uma `string` (texto). Se digitar números aqui, eles continuam como texto!

2. `number()` - Entrada Numérica Real

Evita ter que converter strings com `Number()`. Já entrega o tipo numérico pronto:

```
import { number } from '@inquirer/prompts';

const idade = await number({
  message: 'Qual é a sua idade?',
  min: 0,
  max: 120,
  step: 1 // Use step: 'any' quando quiser aceitar números com ponto decimal
});

console.log(`Idade registrada: ${idade} | Tipo de dado: ${typeof idade}`);
```

★ Vantagem: O próprio terminal já recusa se o aluno tentar digitar letras ou valores fora do `min/max` !

3. `confirm()` - Pergunta Booleana (Sim / Não)

Ideal para confirmações diretas e decisões de fluxo:

```
import { confirm } from '@inquirer/prompts';

const confirmarMatricula = await confirm({
  message: 'Deseja confirmar sua inscrição na turma?',
  default: true // Se o usuário der apenas Enter, assume Sim
});

console.log(`Resposta: ${confirmarMatricula} | Tipo: ${typeof confirmarMatricula}`);
```



Como responder no teclado: Digite `y` (yes) ou `s` (sim) para `true`, ou `n` (no) para `false`.

4. `select()` - Menu de Opções com Setas

Cria menus interativos profissionais para o terminal:

```
import { select } from '@inquirer/prompts';

const turno = await select({
  message: 'Selecione o seu turno de estudo:',
  choices: [
    { name: 'Manhã (08h às 12h)', value: 'M' },
    { name: 'Tarde (13h às 17h)', value: 'T' },
    { name: 'Noite (18h às 22h)', value: 'N' }
  ]
});

console.log(`Turno selecionado: ${turno}`);
```

🎮 O aluno navega com as setas `↑` e `↓` do teclado e confirma com `Enter`. O programa recebe o `value` !

5. `checkbox()` · Seleção Múltipla

Permite que o usuário marque uma ou mais opções da lista:

```
import { checkbox } from '@inquirer/prompts';

const modulos = await checkbox({
  message: 'Quais competências você deseja desenvolver?',
  choices: [
    { name: 'Lógica e Algoritmos com JavaScript', value: 'logica', checked: true },
    { name: 'Banco de Dados Relacional (SQLite)', value: 'sql' },
    { name: 'Interfaces Desktop com Electron', value: 'electron' }
  ]
});

console.log('Módulos selecionados:', modulos);
// Retorna uma lista (Array): [ 'logica', 'sql' ]
```

 Controle no Teclado: Aperte **Espaço** para marcar/desmarcar e aperte **Enter** para confirmar.

6. password() · Entrada Oculta para Senhas

Fundamental para evitar que senhas e credenciais apareçam no monitor ou telão:

```
import { password } from '@inquirer/prompts';

const senha = await password({
  message: 'Crie uma senha de acesso ao sistema:',
  mask: '*' // Substitui cada caractere digitado por um asterisco
});

console.log("Senha armazenada com sucesso na memória!");
```

🔒 **Segurança:** O texto real fica guardado na variável, mas nenhum colega sentado ao lado consegue ler o que foi digitado!

Fazendo a Máquina Calcular: Os 6 Operadores

JavaScript possui operadores matemáticos nativos para processamento numérico:

Operador	Operação	Exemplo em Código	Resultado
<code>+</code>	Adição	<code>10 + 5</code>	<code>15</code>
<code>-</code>	Subtração	<code>20 - 8</code>	<code>12</code>
<code>*</code>	Multiplicação	<code>7 * 6</code>	<code>42</code>
<code>/</code>	Divisão	<code>25 / 2</code>	<code>12.5</code>
<code>%</code>	Módulo (<i>Resto da Divisão</i>)	<code>10 % 3</code>	<code>1</code> (<i>resto!</i>)
<code>**</code>	Exponenciação (<i>Potência</i>)	<code>2 ** 3</code>	<code>8</code> (2^3)



O Truque do Módulo: `numero % 2 === 0` verifica se o número é **PAR**! Se sobrar `1`, ele é **ÍMPAR**.

A Armadilha Histórica da Soma vs Concatenação

O que acontece quando juntamos texto e número?

```
// O sinal de '+' é sobrecarregado: ele SOMA números, mas CONCATENA textos!  
console.log("20" + 5); // "205" → ERRO COMUM! Juntou os textos!  
console.log("20" - 5); // 15 → O JS converte automaticamente na subtração!  
console.log("10" * 3); // 30 → Converte na multiplicação!
```

Como Garantir Números Seguros:

1. Usando `number()` do Inquirer: já devolve o tipo `number` garantido.
2. Usando `Number(texto)`: converte strings numéricas de forma estrita:

```
const textoIdade = "25";  
const idadeReal = Number(textoIdade); // Agora é o número 25!  
console.log(idadeReal + 5); // 30 (Soma matemática correta)
```

Comparando Valores: Verdadeiro ou Falso?

Operadores relacionais fazem perguntas à máquina e **sempre devolvem um booleano** (`true` ou `false`):

Operador	Significado	Exemplo	Retorno
<code>===</code>	Estritamente Igual (<i>valor E tipo</i>)	<code>10 === 10</code>	<code>true</code>
<code>!==</code>	Estritamente Diferente	<code>10 !== "10"</code>	<code>true</code> (<i>tipos diferentes!</i>)
<code>></code>	Maior que	<code>18 > 16</code>	<code>true</code>
<code><</code>	Menor que	<code>25 < 10</code>	<code>false</code>
<code>>=</code>	Maior ou Igual a	<code>18 >= 18</code>	<code>true</code>
<code><=</code>	Menor ou Igual a	<code>7 <= 5</code>	<code>false</code>

🚫 Regra de Ouro Senac: NUNCA use `=` ou `≠`. Eles tentam adivinhar tipos e criam bugs ocultos. Use sempre a igualdade estrita `===` e `!==` !

Combinando Condições: E, OU e NÃO

No mundo real, decisões dependem de várias condições simultâneas:

&& (E / AND)

Todas as condições devem ser verdadeiras.

`(idade ≥ 18 && temCnh)`

|| (OU / OR)

Pelo menos uma condição deve ser verdadeira.

`(temIngresso || nomeNaLista)`

! (NÃO / NOT)

Inverte o valor lógico.

`!chovendo` vira `true` se estava falso.

```
const idade = 19;  
const temIngresso = true;  
const acompanhado = false;
```

// Regra: Pode entrar se tiver ingresso E (for maior de idade OU estiver acompanhado)

```
const podeEntrar = temIngresso && (idade ≥ 18 || acompanhado);  
console.log(`Acesso liberado? ${podeEntrar}`); // true
```

Mini-Desafio: Simulador de Matrícula Senac

Crie um arquivo chamado `simulador_matricula.js`:

1. Peça o nome do aluno via `input`.
2. Peça a idade via `number` (com `min: 0, max: 120`).
3. Pergunte se possui o ensino fundamental completo via `confirm`.
4. Calcule a regra de elegibilidade do curso Programador de Sistemas:
 - Regra: Idade mínima de 16 anos E escolaridade completa.
5. Imprima no terminal o resumo com o resultado da expressão lógica:

```
=====
VALIDADOR DE MATRÍCULA SENAC
=====
Candidato(a): Marcus Vinicius
Idade informada: 17 anos
Escolaridade confirmada? true
-----
Elegível para o curso? true
```

SÍNTESE & ENCERRAMENTO

O Que Conquistamos na Aula 02?

1. Entrada & Saída

Formatação com `repeat`, `padEnd` e captura tipada com `@inquirer/prompts`.

2. Memória & Tipos

Declarações seguras com `const`, inspeção com `typeof` e conversão com `Number()`.

3. Lógica Computacional

Operações aritméticas (`+` `-` `*` `/` `%`), relacionais (`===` `!==` `>` `<`) e lógicas (`&&` `||` `!`).

PRÓXIMA AULA · AULA 03

Estruturas de Decisão

Ensinando o Computador a Escolher Caminhos

Bifurcações na execução com if, else, else if, o poder do switch...case e o atalho do operador ternário.

Excelente prática e nos vemos na Aula 03! 🚀